Software Engineering - Lab 1: Version Control with Git

Objective: The goal of this assignment is to familiarize yourself with the basic workflow of Git, a powerful version control system. You will practice initializing a repository, making commits, creating branches, handling merge conflicts, reverting changes, and pushing your work to a remote repository.

Context: We will be working with a simple web-based calculator. The focus of this assignment is on using Git to manage the project's source code, not on the programming itself. All necessary code changes will be provided.

Prerequisites

1. **Git Installed:** You must have Git installed on your computer. You can download it from git-scm.com.
2. **A GitHub Account:** You will need a free account on GitHub to host your remote repository.
3. **Project Files:** Download the calculator folder which contains index.html, style.css, and script.js.

Submission

To complete this assignment, you must submit two items to the course learning portal. Please adhere strictly to the naming conventions below to ensure your submission is processed correctly.

1. **Public GitHub Repository URL:**
   - Your remote repository on GitHub must be public so that it can be accessed for grading.
   - The repository must be named according to the following format: `git-calculator-<StudentID>-<YourName>`
     - **Example:** `git-calculator-20012345-NguyenVanAn`
   - Submit the full URL to this repository (e.g., `https://github.com/your-username/git-calculator-20012345-NguyenVanAn`).
2. **Commit History Log File:**
   - You must generate a text file containing the complete output of the `git log` command from your local repository after all steps have been completed.
   - The file must be named according to the following format: `git_log_<StudentID>_<YourName>.txt`
     - **Example:** `git_log_20012345_NguyenVanAn.txt`
   - To generate the file, you can run the following command in your terminal: `git log > git_log_<StudentID>_<YourName>.txt`

Part 1: Setting Up The Local Repository

The first step is to create a local Git repository for the calculator project.

1. **Navigate to your project folder:** Open your terminal or command prompt and navigate into the calculator directory you downloaded.
2. **Initialize the repository:** Run the following command to turn this directory into a Git repository.
   ```
   git init -b main
   ```

   Use the `-b main` flag to initialize repository with main as default branch.

3. **Check the status:** See the state of your new repository.
   ```
   git status
   ```

   You will see that the files are "untracked."
4. **Stage the files:** Add all the files in the current directory to the staging area.
   ```
   git add .
   ```

5. **Make your first commit:** Commit the staged files to the project history.
   ```
   git commit -m "Initial commit: Add base calculator project files"
   ```

6. **Review the history:** Look at the commit history for your repository.
   ```
   git log
   ```

   You should see your first commit, along with your author information and the commit message.

Part 2: Feature Development with Branching

In Git, branches are used to develop features in isolation. Let's create a new feature: a "Square Root" (√) button.

1. **Create and switch to a new branch:** Create a new branch called feature-sqrt and switch to it.
   ```
   git switch -c feature-sqrt
   ```

   (Note: This is a shortcut for git branch feature-sqrt followed by git switch feature-sqrt)
2. **Implement the feature:**
   - **Modify index.html:** Open index.html. We need to add a button for the square root function. Find the line with the percent button and add the new button right after it, but before the divide button.
     Find this block:
```
<button class="key util" data-action="percent" aria-label="Percent">%</button>
<button class="key op" data-op="/" aligned="Divide">÷</button>
```

     And modify it to look like this:

```
<button class="key util" data-action="percent" aria-label="Percent">%</button>
<button class="key util" data-action="sqrt" aria-label="Square root">√</button>
<button class="key op" data-op="/" aria-label="Divide">÷</button>
```

- **Modify script.js:** Open script.js. We need to add the logic for the square root function.
  First, find this function block:
  ```
  function inputPercent() {
    const value = parseFloat(state.displayValue);
    if (!Number.isNaN(value)) {
      state.displayValue = formatNumber(value / 100);
      setDisplay(state.displayValue);
    }
  }
  ```
  Add this new function directly below it:
  ```
  function inputSqrt() {
    const value = parseFloat(state.displayValue);
    if (value >= 0) {
      state.displayValue = formatNumber(Math.sqrt(value));
      setDisplay(state.displayValue);
    } else {
      showError("Invalid input for sqrt");
    }
  }
  ```
  Next, find this line in the event listener near the end of the file:
  ```
  if (btn.dataset.action === 'percent') return inputPercent();
  ```
  Add this new line directly below it:
  ```
  if (btn.dataset.action === 'sqrt') return inputSqrt();
  ```

3. **Commit the feature:** Stage and commit the changes on the feature-sqrt branch.
   ```
   git add .
   git commit -m "Feature: Add square root functionality"
   ```

Part 3: Handling a Merge Conflict

Merge conflicts happen when Git cannot automatically resolve differences in code between two branches. This is a common and important scenario to learn how to handle.

1. **Switch back to the main branch:**
   ```
   git switch main
   ```

2. **Simulate a conflicting change:** While you were working on the sqrt feature, imagine a teammate changed the layout in index.html on the main branch. Open index.html and modify the same section where you added your button.
   Change this:

```
<button class="key util" data-action="percent" aria-label="Percent">%</button>
<button class="key op" data-op="/" aligned="Divide">÷</button>
```

To this (reordering the buttons):

```
<button class="key op" data-op="/" aria-label="Divide">÷</button>
<button class="key util" data-action="percent" aria-label="Percent">%</button>
```

3. **Commit the conflicting change:** Stage and commit this change on the main branch.
   ```
   git add index.html
   git commit -m "Refactor: Reorder operator keys"
   ```

4. **Attempt the merge:** Now, try to merge your feature-sqrt branch into main.
   ```
   git merge feature-sqrt
   ```

   Git will fail and report a **merge conflict** in index.html.
5. **Resolve the conflict:**
   - Open index.html in your editor. You will see conflict markers added by Git:
     ```
     <<<<<<< HEAD
     <button class="key op" data-op="/" aria-label="Divide">÷</button>
     <button class="key util" data-action="percent" aria-label="Percent">%</button>
     =======
     <button class="key util" data-action="percent" aria-label="Percent">%</button>
     <button class="key util" data-action="sqrt" aria-label="Square root">√</button>
     <button class="key op" data-op="/" aria-label="Divide">÷</button>
     >>>>>>> feature-sqrt
     ```

   - HEAD refers to the main branch, and feature-sqrt refers to the incoming branch.
   - **Edit the code** to resolve the conflict. You need to decide what the final code should look like. Let's decide to keep all three buttons in the desired order. Delete the conflict markers and arrange the code as follows:
     ```
     <button class="key util" data-action="percent" aria-label="Percent">%</button>
     <button class="key util" data-action="sqrt" aria-label="Square root">√</button>
     <button class="key op" data-op="/" aria-label="Divide">÷</button>
     ```

6. **Complete the merge:**
   - Stage the resolved file:
     ```
     git add index.html
     ```

   - Commit the merge. Git will pre-populate a commit message for you.
     ```
     git commit
     ```

     (This will open your default text editor to confirm the message. Just save and close it.)
7. **Clean up:** Delete the feature branch, as it's now merged.
   ```
   git branch -d feature-sqrt
   ```

Part 4: Ignoring Files with .gitignore

Not all files should be tracked by Git (e.g., log files, build output).

1. **Create a dummy file:**
   `touch debug.log`

2. **Check status:** Run git status. You'll see debug.log is untracked.
3. **Create and configure .gitignore:** Create a new file named .gitignore and add the following line to it:
   `debug.log`

4. **Check status again:** Run git status again. debug.log is now hidden, but the new .gitignore file appears.
5. **Commit the .gitignore file:** It's important to track this file in your repository.
   ```
   git add .gitignore
   git commit -m "Build: Add .gitignore to ignore log files"
   ```

Part 5: Reverting a Commit

Sometimes you need to undo a previous commit. git revert is a safe way to do this, as it creates a new commit that reverses the specified changes.

1. **Find a commit to revert:** Let's decide the "Refactor: Reorder operator keys" commit was a mistake. First, find its hash.
   ```
   git log --oneline
   ```

   Copy the 7-character hash for that specific commit.
2. **Revert the commit:** Use the hash you copied.
   `git revert <your-commit-hash-here>`

3. After the command, you will be notified of a conflict in the `index.html` file. Resolve the conflict following guidance from Part 3. Then, stage the resolved `index.html` file.
   `git add index.html`

4. Run the revert command
   `git revert --continue`

   An editor will open for the revert commit message. You can leave the default message, save, and close.
5. **Verify:** Check git log again. You'll see a new "Revert..." commit. If you inspect index.html, you will see that the button order has been reverted.

Part 6: Working with Remotes

A remote repository (e.g., on GitHub) is for collaboration and backup.

1. **Create a repository on GitHub:** Go to GitHub, log in, and create a new, empty repository. **Do not** initialize it with a README. Name it `git-calculator-<StudentID>-<YourName>`.

2. **Link your local repo to the remote:** GitHub will provide a URL for your new repository. Copy it. Back in your terminal, run this command, replacing **<URL>** with yours.
   `git remote add origin <URL>`

3. **Push your code:** Send your local commit history up to the remote repository.
   `git push -u origin main`

   (Use master if that's your default branch name).
4. **Verify:** Refresh your GitHub repository page. You should see all your project files and the full commit history.

Grading Rubric

Part	Criteria	Points
1. Repository Setup	-All three base project files (`index.html`, `style.css`, `script.js`) are present in the initial commit.	7.5
	The commit message is descriptive and accurate (e.g., `"Initial commit: Add base calculator project files"`).	7.5
	Subtotal	**15**
2. Feature Development & Branching	A new branch named `feature-sqrt` is created and used for all feature-related changes.	5
	The square root button (√) is correctly added to `index.html` in the specified location.	5
	The `inputSqrt()` function and the corresponding event listener logic are correctly added to script.js.	5
	The changes to both `index.html` and `script.js` are committed together on the `feature-sqrt` branch with a clear message (e.g., `"Feature: Add square root functionality"`).	5
	Subtotal	**20**
3. Merge Conflict Resolution	The conflicting change (reordering buttons) is correctly made and committed to the main branch before the merge attempt.	5
	The commit log clearly shows a merge commit where `feature-sqrt` was merged into `main`.	5
	The merge conflict in `index.html` is resolved correctly. The final code includes the percent, square root, and divide buttons in the correct, combined order.	10
	The `feature-sqrt` branch is deleted after it has been successfully merged into `main`.	5
	Subtotal	**20**
4. Using .gitignore	A `.gitignore` file is created in the root directory and contains the entry debug.log.	5

	The `.gitignore` file itself is added and committed to the repository with a descriptive message.	5
	Subtotal	**10**
5. Reverting a Commit	The log shows a new commit that explicitly reverts the `"Refactor: Reorder operator keys"` commit, identified by its hash.	5
	Any conflict arising during the revert process is resolved correctly, as indicated by the final state of `index.html`.	5
	The content of `index.html` after the revert commit correctly reflects the state of the code before the reverted change was made.	5
	Subtotal	**20**
6. Remote Repository & Submission	The remote repository on GitHub is public, correctly named, and the final history from the local main branch has been successfully pushed.	10
	Both the public GitHub URL and the correctly named `git_log` text file are submitted as required.	5
	Subtotal	**15**
	TOTAL	**100**

Good luck!